Date __/__/__

Network Denial of service (DOS) & Amplification Attacks

Denial of Service: take a service offline by exhausting it's resource. so, legitimate users cannot access it.

The defining characteristic of a DoS attack is "Amplification". An attacker wants to do very small amount of computing work while forcing the victim to do a massive amount of work.

Two types of attacks

1. DoS Bugs (design flaws in protocols)
2. DoS Floods (brute force or botnets)

DoS Bug: TCP SYN Flood

Date ___/___/_____

When client $\xrightarrow{SYN}$ server & server $\xrightarrow{SYN+ACK}$ client has happened,

Server allocates memory for a backlog queue to remember this "half-open" connection while it wait for the client's final ACK.

A single attacker generates SYN packets using completely random, spoofed source IP address. Once the queue is full, Server rejects new connection

Therefore SYN-flood.

Defense: SYN cookies

- Remove state from the server entirely using SYN cookies, no more backlog queues to exploit.

calculate a hash using connection parameters (client IP, Port, server IP, Port & a secret key)

Server embeds this hash into the sequence number of the SYN-ACK packet it sends, & forgets about the connection.

For next ACK, subtract 1, recalculate the hash, if hash matches, it allocates memory & opens the connection.

Now to utilize the same strategy, botnet needs to do the 3-way handshake first. meaning revealing its true IP.

Defending proxy or firewall can simply log those IP's & permanently block them.

Date __/__/____

Application layer DoS

If the attacker completes the handshake they move up the stack to the Application layer.

Goal is to exhaust server's CPU or database rather than network bandwidth

Ex: Large File Request: botnet sends an HTTP get request demanding a large file. The server streams the file to the network

Ex 2: An Attacker can initiate TLS handshakes, forcing server to do cryptographic math in large amounts until CPU hits 100% & it crashes.

Date ___/___/___

Defense?

(i) Client Puzzles (Proof of Work)

stop accepting free requests, instead force every client to ~~do a~~ solve a math puzzle (like finding SHA-1 hash collision) before it will process HTTP request or TLS Handshake.

No more "Amplification".

2. CAPTCHAS : puzzles only prove the client has CPU power; A Captcha verifies that the connection is driven by an actual human.

Date __/__/____

DNS Reflection Attack :

Attacker spoofs victims IP, sends random DNS queries to servers worldwide.

Now servers send DNS responses to the victim crushing there network.

love bombing ??

The Onion Routing (TOR)

Problem: Even with TLS & encryptions, IP's are in plaintext.

Anyone monitoring the network, ISP, ~~proxy~~ firewall, Bharat Sarkar ... knows who talked to whom.

Date ___/___/_____

Solution: TOR circuit

when using TOR browser, it doesn't send your packets directly to the destination. Instead it downloads a directory of volunteer routing nodes & selects three of them to build a temporary path, a circuit.

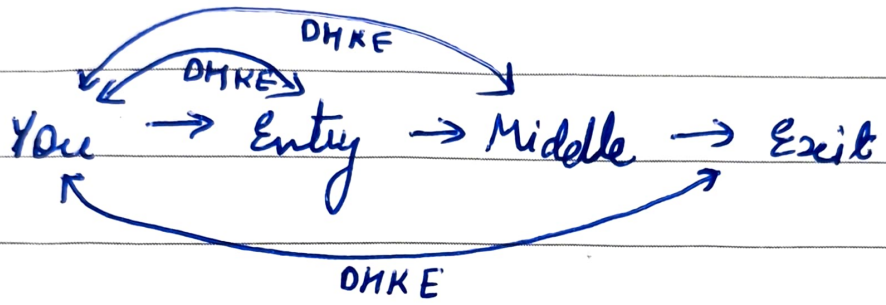
1. The Entry Guard Node
2. The Middle Node
3. The Exit Node

Before sending any data, your machine uses Deffie Hellman to a independently agree on symmetric key with each of the nodes.

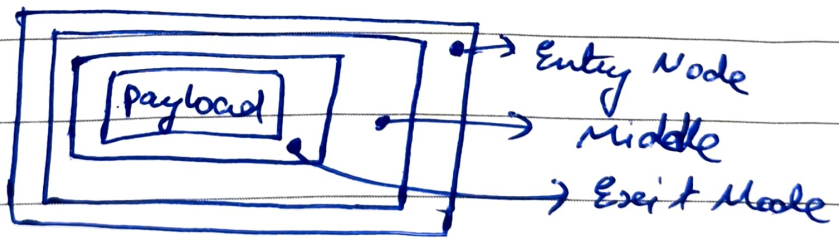
Now, Your computer takes the HTTP request & encrypts it three times, like layers of an onion

Date ___/___/_____

- The core : Encrypted with Exit Node's key
- The middle layer : Middle Node's key
- The Outer layer : Entry Nodes key.



- Entry node knows who you are, but cannot read the ~~des~~ final destination



- Middle Node knows entry & exit nodes, no idea who you are or what the destination is
- Exit Node peels the final layer, it knows the dest. but no idea who you are.